Software Engineering Lab
(MA 616)

Mini Project

Project Title:

SplitPay

Splitting bills without splitting friendships

Team Members:

Aryan Narang (244CA009)
Deeksha Baluni (244CA012)
Harsh Singh (244CA018)
Kanhaiya (244CA022)

Department of Mathematical & Computational Sciences
National Institute of Technology Karnataka, Surathkal, India

Contents

1 Problem Statement **3**

2 Team Members and Roles **3**

3 Project Objective **3**

4 Scope of the Project **3**

5 Target Users & Stakeholders **4**

6 Technology Stack **4**

7 Modules & Major Features **4**

8 Nice-to-Have / Future Scope **4**

9 Tentative Timeline / Work Plan **5**

10 Functional Requirements **5**

11 Non-Functional Requirements **6**

12 External Interfaces **7**

13 Use Case / Scenarios **8**

14 UML Diagrams **11**

15 Modeling Approach **15**

16 Why This Project? **15**

17 Expected Challenges **16**

18 App Screenshots **17**

19 References **30**

1. Problem Statement

Managing shared expenses in real time remains a complex and frustrating challenge, particularly when individual consumption varies significantly. Manual calculations are not only time-consuming, but also prone to errors, while existing expense-splitting applications often demand tedious user inputs or rely on rigid, one-size-fits-all methods that overlook individual usage. These shortcomings often lead to delayed settlements, social discomfort, and unfair cost distribution, especially in common scenarios like group dining, shared shopping, or collective travel. A more intelligent, flexible, and user-friendly solution is clearly needed.

2. Team Members and Roles

- **Aryan Narang (244CA009)** — Team Lead, UI/UX Lead, Coordination, Frontend and Backend Support
- **Deeksha Baluni (244CA012)** — Frontend Developer, Testing and Quality Assurance
- **Harsh Singh (244CA018)** — Frontend Developer, UI Integration and Component Logic
- **Kanhaiya (244CA022)** — Backend Developer, Database and API Management

3. Project Objective

The objective is to develop a clean, intuitive, and AI-powered mobile application designed to streamline and automate the process of splitting shared expenses. The application will incorporate the following key features:

- Seamless Bill Input: Users can conveniently scan or upload both physical and digital bills directly within the app.
- Smart Item Extraction: Utilising Optical Character Recognition (OCR) in combination with advanced Large Language Models (LLMs), the app will accurately extract and organise itemised bill details.
- Individual Item Assignment: Users can assign specific items to group members with ease, allowing for precise and personalised expense attribution.
- Intelligent Bill Splitting: The app will apply adaptive, consumption-based logic to ensure fair and transparent distribution of the total cost, eliminating manual effort, ambiguity, and bias.

4. Scope of the Project

SplitPay will be developed as a mobile-first application tailored to simplify and automate the process of splitting shared expenses in real-life group scenarios. The scope of the project focuses on delivering a seamless, intelligent, and intuitive experience with the following core capabilities:

- **Bill Scanning and Uploads:** Users can scan physical receipts or upload existing digital bills (images/PDFs) directly from their phones, eliminating the need for manual entry.
- **Real-Time Item Extraction:** Leveraging on-device Optical Character Recognition (OCR) and lightweight Large Language Models (LLMs), the app will extract itemized entries from the uploaded bills with high accuracy and minimal delay.
- **Intuitive Item-to-User Assignment:** The extracted items will be presented in a clean UI where users can assign each item to relevant group members using a simple checkbox interface — ensuring fairness and reducing ambiguity.
- **Flexible Bill Splitting Logic:** SplitPay will support multiple cost distribution methods — equal split, per-item basis — making it adaptable to diverse group needs and consumption patterns.
- **Group Creation and Management:** Users can create groups, add or remove members, and assign expenses across different events or occasions. Each group's data will remain separate and persist locally or in the cloud.

- **Comprehensive Expense History:** The app will maintain detailed per-user logs of past expenses, summaries, and owed/settled amounts, giving users full transparency and financial clarity.

5. Target Users & Stakeholders

- **Primary users:** Students, flatmates, friends, colleagues, families
- **Stakeholders:** Development team, course evaluators, and testers

6. Technology Stack

- **Frontend:** Flutter
- **Backend:** Node.js + Express
- **Auth:** JWT
- **Database:** MongoDB (via MongoDB Atlas Free Tier)
- **OCR:** Tesseract.js (Fully free and runs in-browser)/ Google ML Kit
- **LLM:** Ollama with **Gemma** /Google Gemini
- **Hosting:** Google PlayStore (for frontend) + Render / Cyclic.sh (for free backend hosting)

7. Modules & Major Features

- User Authentication & Group Management : Enables secure user login and intuitive group creation with member management features.
- Bill Upload & OCR Parsing : Allows uploading of bill images or PDFs and extracts text using OCR technology.
- LLM Parsing Pipeline : Uses Large Language Models to intelligently convert OCR output into structured itemized data.
- Checkbox-Based Item Assignment UI : Provides an intuitive interface for assigning items to users via checkboxes.
- Smart Split Engine (Equal, Per Item) : Calculates expense shares fairly using multiple customizable splitting methods.
- Fallback Manual Entry Module : Offers a manual input option for bills when automated parsing is insufficient.

8. Nice-to-Have / Future Scope

- Live Cost Tracking : Show each user's share in real-time as items are assigned.
- UPI Integration : Allow direct payments through UPI or digital wallets within the app.
- Group Chat & Notes : Enable in-app communication and shared notes for better coordination.
- Spending Insights : Visualize expenses with category-wise breakdowns and trends.

9. Tentative Timeline / Work Plan

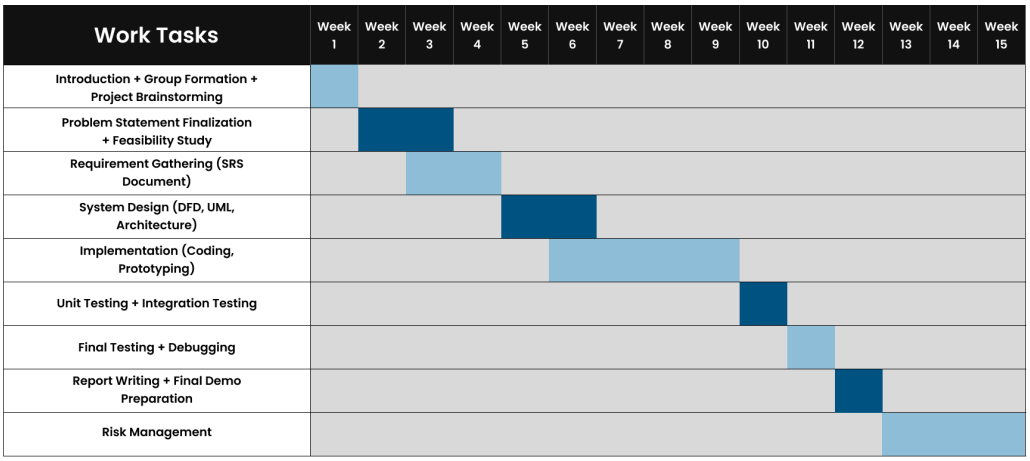

Figure 1: Tentative Work Plan – Gantt Chart

10. Functional Requirements

The SplitPay system shall provide the following functionality:

1. **User Authentication**
   - The system shall allow users to register using email/phone and log in securely.
   - The system shall validate credentials and create a session token for authenticated access.
2. **Group Management**
   - The system shall allow users to create new groups and invite members.
   - The system shall allow group admins to add or remove members.
   - The system shall allow members to leave a group voluntarily.
3. **Bill Upload**
   - The system shall allow users to upload bills in image (JPG, PNG) or PDF format.
   - The system shall accept files up to 5 MB in size.
4. **OCR Processing**
   - The system shall extract raw text from uploaded bills using OCR.
   - The OCR engine shall return item lines with text confidence scores.
5. **LLM Parsing**
   - The system shall process OCR text through an LLM to structure it into items with name, quantity, and price fields.
   - The system shall return bill metadata (total, date, vendor if available).
6. **Item Assignment**
   - The system shall allow users to assign each item to one or more group members via a UI.

- The system shall allow editing, adding, or deleting items manually.

7. **Expense Splitting**
   - The system shall calculate dues using multiple methods: equal split, per-item assignment, and custom percentage split.
   - The system shall generate balances ("You owe" / "You are owed") for each member.
8. **Expense History**
   - The system shall store all expenses per group.
   - The system shall provide users with a history log of past bills, splits, and settlements.
9. **Settlement Recording**
   - The system shall allow members to mark payments as settled.
   - The system shall update balances accordingly.
10. **Notifications**
    - The system shall notify users when they are added to a group, a new bill is uploaded, an item is assigned to them, or a settlement is recorded.
11. **Preferences**
    - The system shall allow users to mute notifications per group.
    - The system shall allow users to set quiet hours during which no push notifications are sent.

11. Non-Functional Requirements

The SplitPay system shall meet the following non-functional requirements:

1. **Performance Requirements**
   - NFR1: OCR + LLM parsing of a bill shall complete within 5 seconds for bills up to 20 items.
   - NFR2: UI interactions shall respond within 150 ms on supported devices.
   - NFR3: The backend shall support at least 100 concurrent users and 50 bill uploads per minute.
2. **Security Requirements**
   - NFR4: User authentication shall use Firebase Authentication with JWT tokens.
   - NFR5: Passwords shall be hashed with bcrypt or equivalent.
   - NFR6: All data in transit shall be encrypted with TLS 1.2 or higher.
   - NFR7: Only group admins may add/remove members.
   - NFR8: Users shall only access groups and expenses they belong to.
3. **Usability Requirements**
   - NFR9: A first-time user shall be able to create a group and split a bill within 5 minutes.
   - NFR10: The system shall comply with WCAG AA color contrast standards.
   - NFR11: The UI shall be operable with one hand on standard smartphones.
4. **Reliability and Availability Requirements**
   - NFR12: The system shall be available 99% of the time during evaluations.
   - NFR13: Failed OCR or LLM calls shall gracefully fallback to manual entry.
   - NFR14: All expense calculations shall maintain precision up to 2 decimal places.
   - NFR15: Data shall be backed up daily and recoverable within 1 hour.
5. **Maintainability and Extensibility Requirements**

- NFR16: The backend shall follow modular architecture (auth, groups, bills, notifications).
- NFR17: Source code shall be documented with inline comments and API documentation.
- NFR18: The system shall allow future integration of UPI payments and spending insights.

6. **Observability Requirements**
   - NFR19: All API requests and errors shall be logged with timestamps.
   - NFR20: Metrics shall be recorded for OCR time, API latency, and notification success rates.
   - NFR21: All errors shall display meaningful messages to users within 2 seconds.

12. External Interfaces

1. **User Interface (UI)**
   - The application shall provide a cross-platform interface built using Flutter.
   - The UI shall consist of the following primary screens:
     (a) Login / Registration
     (b) Group Management (create group, invite members, view members)
     (c) Bill Upload (image/PDF upload, preview)
     (d) Item Assignment (checkbox-based item to user assignment)
     (e) Expense Summary ("You owe" / "You are owed" view)
     (f) History Panel (list of past expenses and settlements)
     (g) Notification Center (alerts for new bills, assignments, and settlements)
   - The UI shall follow a clean blue-and-white theme, with accessibility compliant contrast (WCAG AA).

2. **Software Interfaces (APIs)**
   - **OCR Interface:** Input: Bill image (JPG/PNG) or PDF. Output (example JSON):

```
{
  "lines": [
    {"text": "Paneer Tikka 180", "confidence": 0.94},
    {"text": "Butter Naan 40", "confidence": 0.89}
  ]
}
```

   - **LLM Parsing Interface:** Input: Raw OCR text. Output (structured JSON):

```
{
  "vendor": "ABC Restaurant",
  "date": "2025-08-20",
  "items": [
    {"name": "Paneer Tikka", "qty": 1, "price": 180},
    {"name": "Butter Naan", "qty": 2, "price": 80}
  ],
  "total": 260
}
```

   - **Backend REST APIs:**
     - `POST /auth/register` – Register user.
     - `POST /groups` – Create a group.
     - `POST /bills/upload` – Upload a bill.
     - `POST /bills/:id/assign` – Assign items to users.
     - `GET /expenses/:groupId` – Retrieve group expenses.
     - `POST /settlements` – Record a settlement.

- `GET /notifications` – Retrieve user notifications.

3. **Database Interfaces**
   - The backend shall use MongoDB Atlas as the database.
   - Example schema (bill record):

```
{
  "_id": "b123",
  "groupId": "g45",
  "uploaderId": "u12",
  "items": [
    {"name": "Paneer Tikka", "assignees": ["u12"], "price": 180},
    {"name": "Butter Naan", "assignees": ["u13","u14"], "price": 80}
  ],
  "total": 260,
  "createdAt": "2025-08-20T15:30:00Z"
}
```

4. **Notification Interfaces**
   - Notifications shall be delivered via Firebase Cloud Messaging (FCM).
   - Example payload:

```
{
  "title": "New Bill Uploaded",
  "body": "Harsh added a bill in Friday Dinner Group",
  "data": {
    "groupId": "g45",
    "expenseId": "e77",
    "screen": "expense"
  }
}
```

   - Users may view notifications in-app under the Notification Center.

13. Use Case / Scenarios

1. **Overview** This section describes the primary interactions between users and the SplitPay system, focusing on the end-to-end flow from bill upload to settlements.
2. **Actors**
   - **User**: Any authenticated member using the app.
   - **Group Admin**: A user with permissions to manage group members.
   - **System Services**: OCR Engine and LLM Parser (background services).
3. **Detailed Use Cases**

UC-01: Create / Join Group

Primary Actor	User / Group Admin
Goal	Create a new group or join an existing one.
Preconditions	User is authenticated.
Trigger	User selects "Create Group" or enters invite code.
Main Flow	1) User enters details. 2) System validates. 3) If new: system creates group, sets creator as admin. 4) If join: system adds user to group.
Alternate Flows	Invalid invite code → error shown.
Postconditions	Group created or user added to group.

UC-02: Upload Bill and Parse

Primary Actor	User
Goal	Upload bill and extract items.
Preconditions	User is member of a group; bill available.
Trigger	User taps "Upload Bill".
Main Flow	1) User selects file. 2) OCR extracts text. 3) LLM structures into items. 4) System returns itemized list.
Alternate Flows	Low OCR confidence → manual edit. Parse failure → manual entry mode.
Postconditions	Itemized bill stored.

UC-03: Assign Items and Calculate Split

Primary Actor	User
Goal	Assign items and compute dues.
Preconditions	Parsed bill available; group has members.
Trigger	User opens item assignment.
Main Flow	1) User assigns members per item. 2) User selects split method. 3) System validates totals. 4) System calculates dues.
Alternate Flows	Invalid custom % → system blocks.
Postconditions	Balances stored (You owe / You are owed).

UC-04: Record Settlement

Primary Actor	User
Goal	Record a payment between members.
Preconditions	Outstanding balances exist.
Trigger	User taps "Settle".
Main Flow	1) User enters payer, payee, amount. 2) System validates. 3) Settlement recorded and balances updated. 4) Notifications sent.
Alternate Flows	Amount exceeds due → system rejects.
Postconditions	Settlement stored, balances updated.

UC-05: View History & Notifications

Primary Actor	User
Goal	Review expenses, settlements, alerts.
Preconditions	User belongs to a group with activity.
Trigger	User opens History/Notifications.
Main Flow	1) System lists past activity. 2) Unread notifications displayed. 3) User deep-links into details.
Alternate Flows	No history → show empty state.
Postconditions	User sees past records; can act on alerts.

UC-06: Manage Members (Admin)

Primary Actor	Group Admin
Goal	Add/remove members; transfer admin.
Preconditions	Group exists; actor is admin.
Trigger	Admin opens Members screen.
Main Flow	1) Admin adds/removes/updates members. 2) System updates membership and roles. 3) Notifications sent.
Alternate Flows	Last admin leaving → force reassignment.
Postconditions	Membership updated successfully.

UC-07: Edit Items / Manual Entry

Primary Actor	User
Goal	Fix OCR/LLM results or enter manually.
Preconditions	Bill uploaded.
Trigger	User taps Edit Items.
Main Flow	1) User edits or adds items. 2) System validates totals. 3) Corrected list stored.
Alternate Flows	Totals mismatch → confirm override.
Postconditions	Clean item list stored.

UC-08: Notification Preferences

Primary Actor	User
Goal	Control notification categories and quiet hours.
Preconditions	User authenticated.
Trigger	User opens Notification Settings.
Main Flow	1) User toggles categories. 2) Sets quiet hours. 3) System enforces preferences.
Postconditions	Notifications delivered per preferences.

UC-09: Search & Export History

Primary Actor	User
Goal	Find past expenses and export summary.
Preconditions	History exists.
Trigger	User applies filters in History.
Main Flow	1) User filters by date/member/amount. 2) Results shown. 3) User exports CSV/PDF.
Alternate Flows	No results → system shows empty state.
Postconditions	Filtered view and export generated.

14. UML Diagrams

This section provides the UML diagrams created for the SplitPay system, which describe the system from multiple perspectives.

1. **Use Case Diagram**

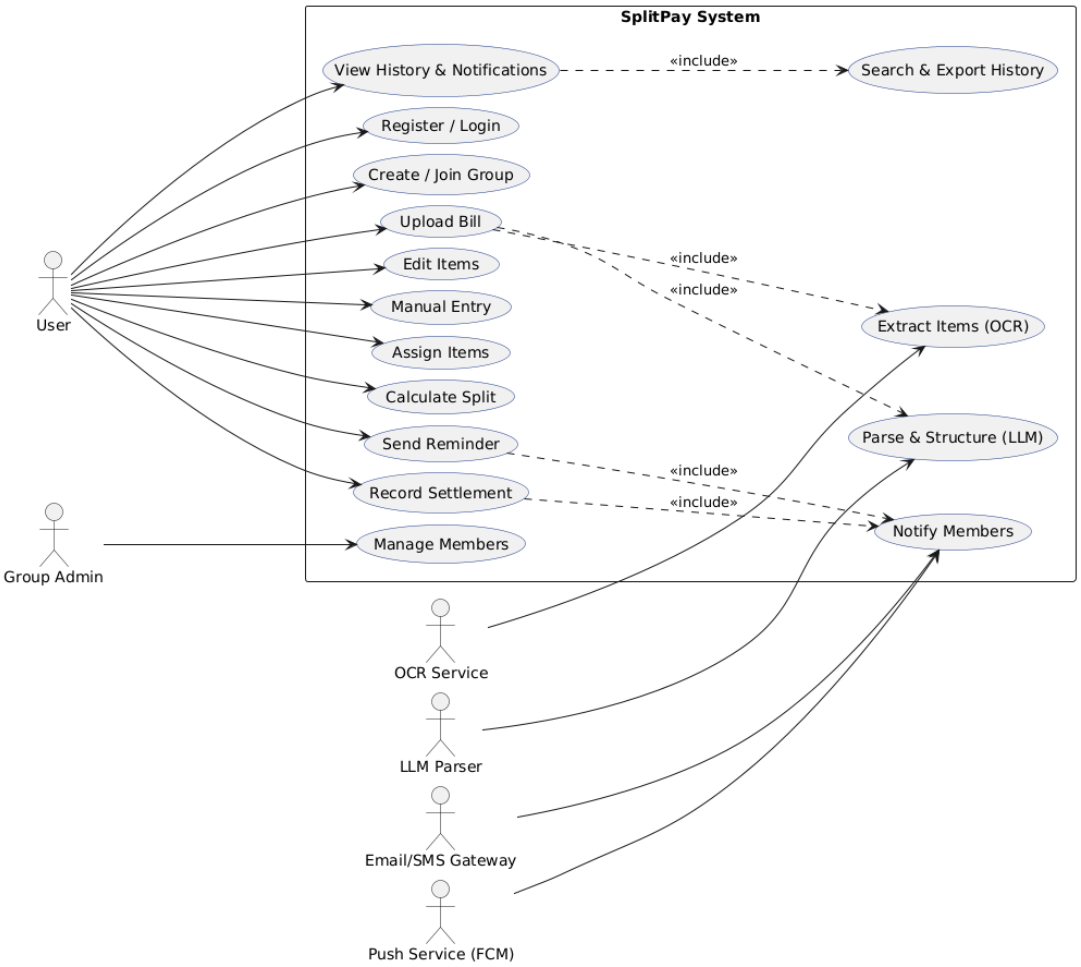

Figure 2: Use Case Diagram for SplitPay

2. Class Diagram

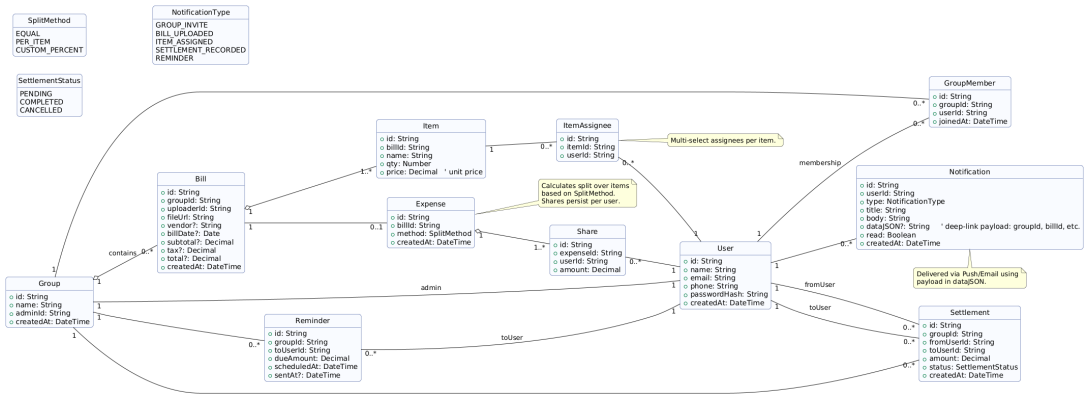

Figure 3: Class Diagram for SplitPay

3. **Activity Diagram**

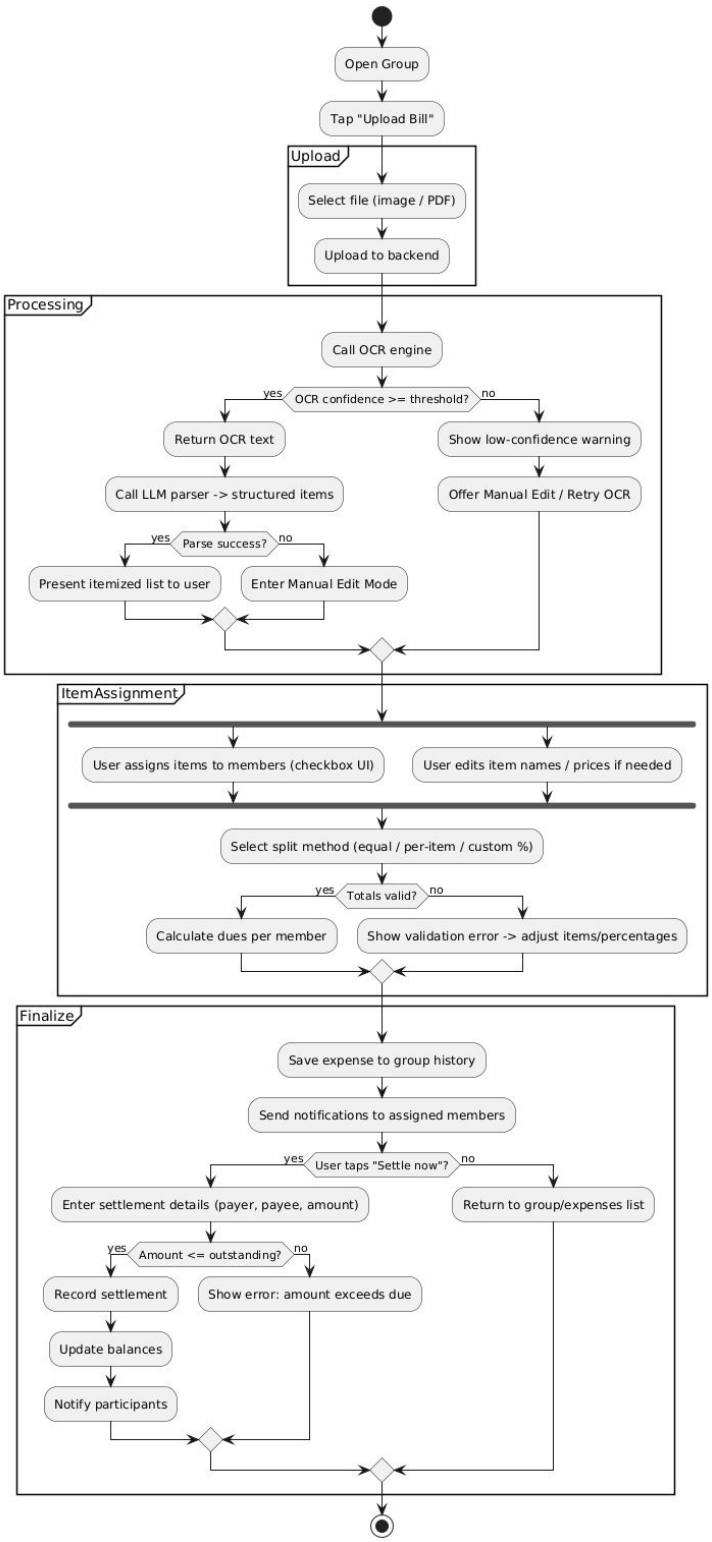

Figure 4: Activity Diagram for Bill Upload and Split Flow

4. Sequence Diagram

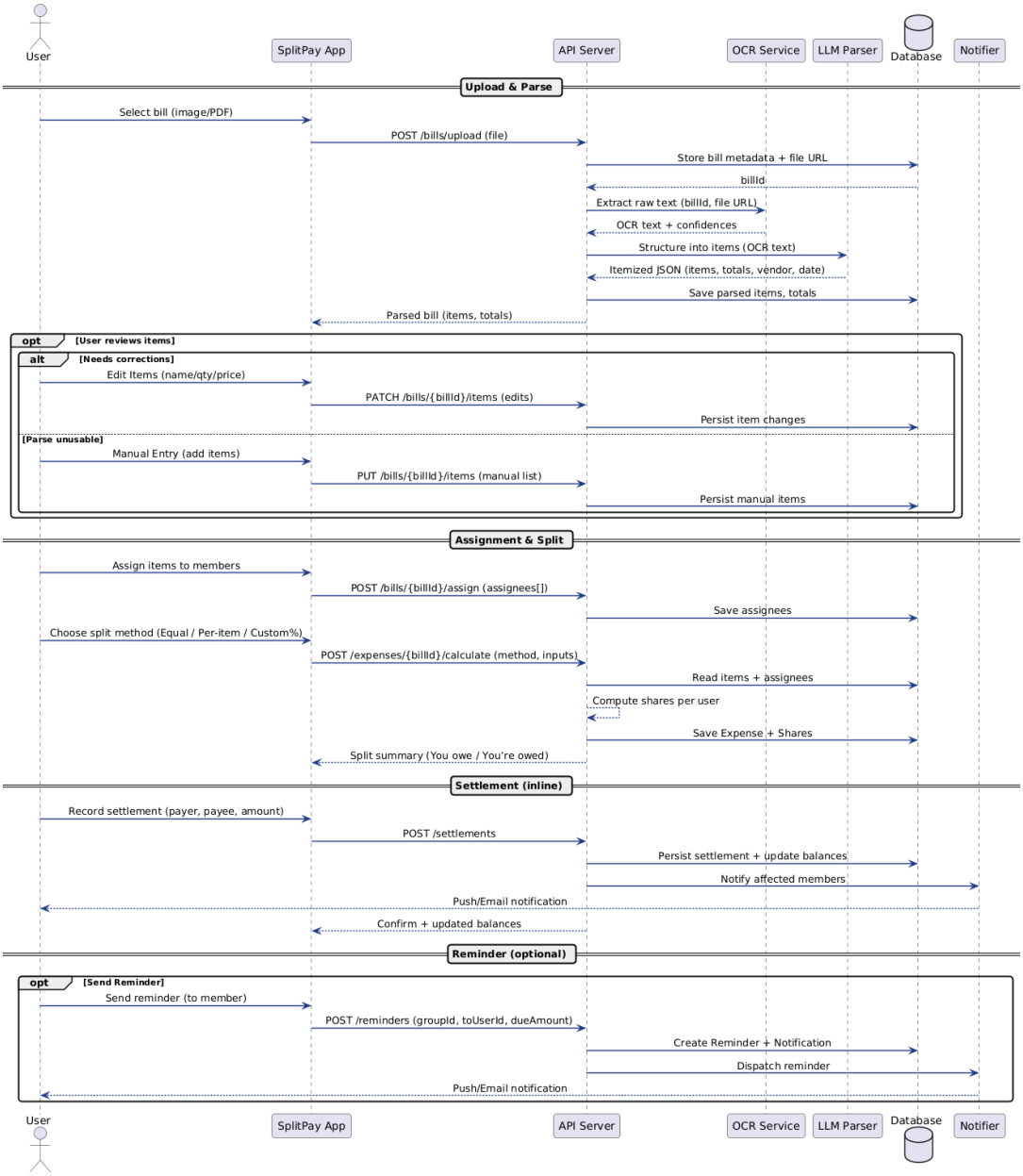

Figure 5: Sequence Diagram for Bill Upload and Split Flow

5. **Summary**
   - The **Use Case Diagram** shows interactions between actors (User, Group Admin, System Services) and the system.
   - The **Class Diagram** models entities such as User, Group, Bill, Item, Expense, and Settlement, along with their relationships.
   - The **Sequence Diagram** illustrates the dynamic flow of uploading a bill, running OCR/LLM parsing, assigning items, and saving expenses.

15. Modeling Approach

The modeling of the SplitPay system was carried out using a combination of **scenario-based** and **class-based** approaches, supported by **UML diagrams**.

- **Scenario-Based Modeling:** Use Case diagrams and detailed use case descriptions were developed to capture the interactions between actors (User, Group Admin, System Services) and the system. These scenarios illustrate how major tasks such as creating groups, uploading bills, parsing items, splitting expenses, and recording settlements are carried out in practice. This approach ensures that user workflows are clearly represented and aligned with requirements.
- **Class-Based Modeling:** The Class Diagram was used to represent the static structure of the system, including key entities (User, Group, Bill, Item, Expense, Settlement, Notification) and their relationships. This helps ensure data integrity, consistency, and a clear separation of concerns in the backend architecture.
- **Behavioral Modeling:** Sequence and Activity Diagrams were created to model the dynamic behavior of the system. The Sequence Diagram captures the interaction flow during bill upload and splitting. These diagrams highlight the order of operations, control flows, and dependencies.
- **Iterative Refinement:** Modeling followed an iterative approach. Initial sketches were created during brainstorming, then refined into UML diagrams as the requirements became clearer. This ensured that the models remain flexible and can evolve with the project.

Overall, the modeling approach provided both a high-level understanding of the system's interactions and a detailed view of its internal structure. This dual perspective supports design, implementation, and testing activities, ensuring the system aligns with user needs and technical feasibility.

16. Why This Project?

Splitting shared expenses is one of those deceptively simple problems that often turns chaotic in real-life situations. Whether it's a dinner, a trip, or a group purchase, the process quickly becomes awkward, inaccurate, and time-consuming. Existing apps either oversimplify the problem or demand excessive manual input.

This project addresses that gap with a smarter, more thoughtful approach—combining modern technologies like OCR and LLMs with clean, intuitive design to solve a real-world problem encountered by nearly every student and social group. It's not just about fairness; it's about making the entire process fast, seamless, and even enjoyable.

Technically, the project is a complete end-to-end system. It spans real-time data extraction, intelligent logic for cost distribution, frontend/backend integration, and user-friendly interfaces. It challenges the team across system architecture, AI integration, and human-centered design—while delivering a genuinely useful product.

That blend of real-world impact, full-stack complexity, and creative problem-solving makes this an ideal fit for a semester-long software engineering project.

17. Expected Challenges

- Low-resolution or crumpled bills reduce OCR accuracy
- Misinterpretation or hallucination by LLMs during receipt parsing
- Designing a UI for item-user assignment that remains clear and uncluttered
- Ensuring group data syncs in real-time without conflicts or lag
- Maintaining app responsiveness across devices with a smooth user experience

18. App Screenshots

This section showcases screenshots from the SplitPay mobile application to illustrate its core interface and user experience.

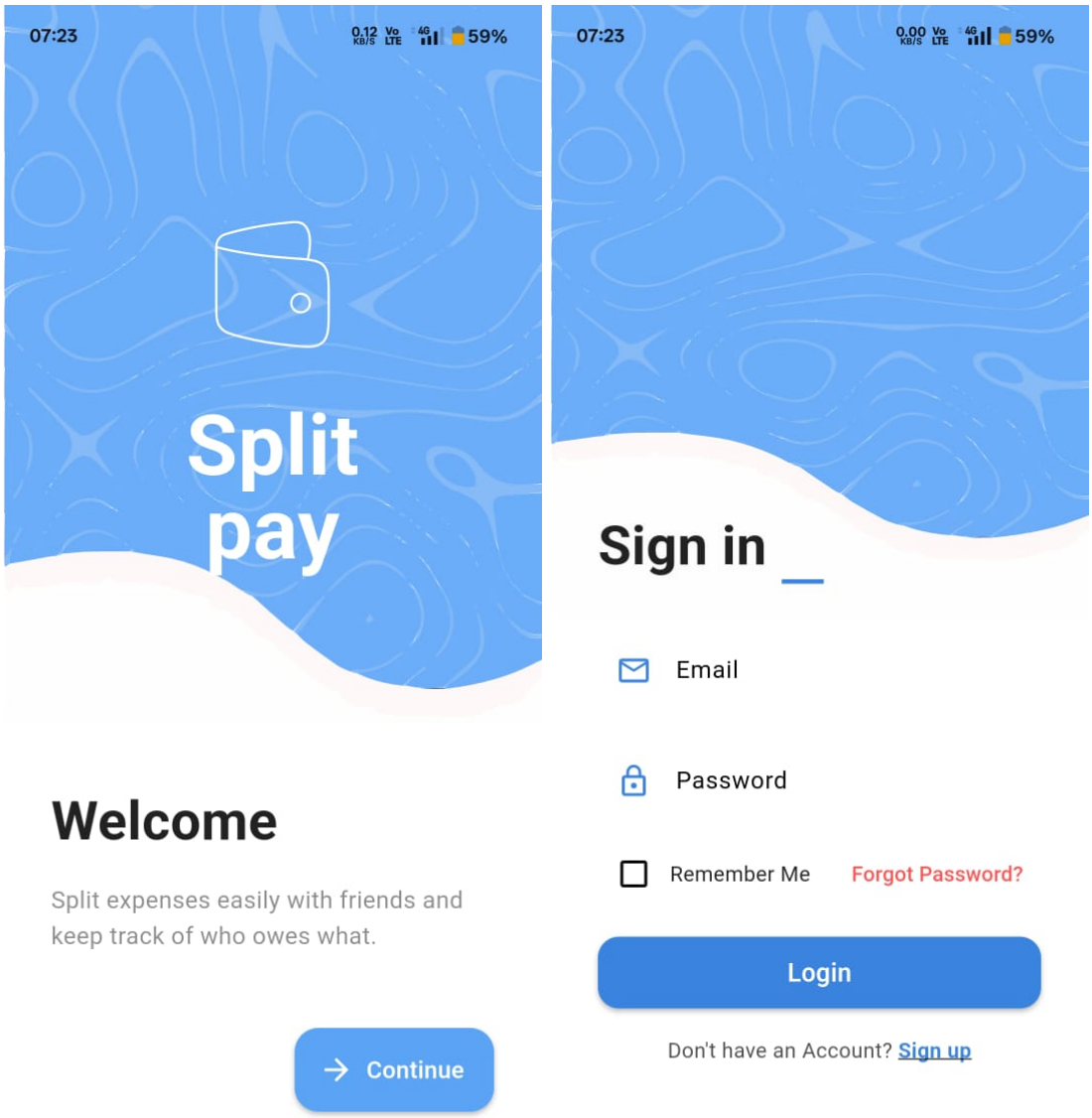

Figure 6: Starting Page and Sign up page

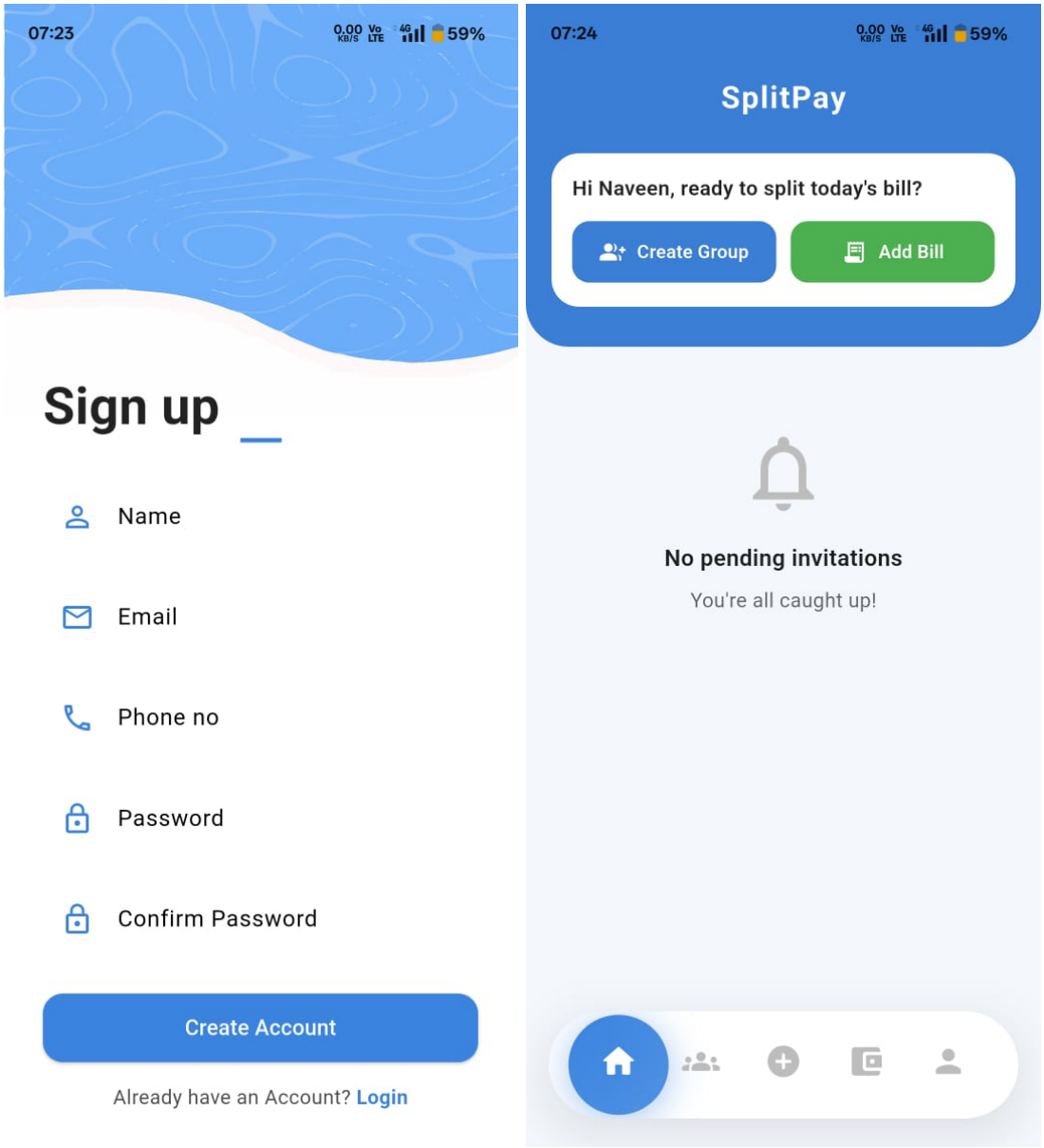

Figure 7: Sign up and Homepage

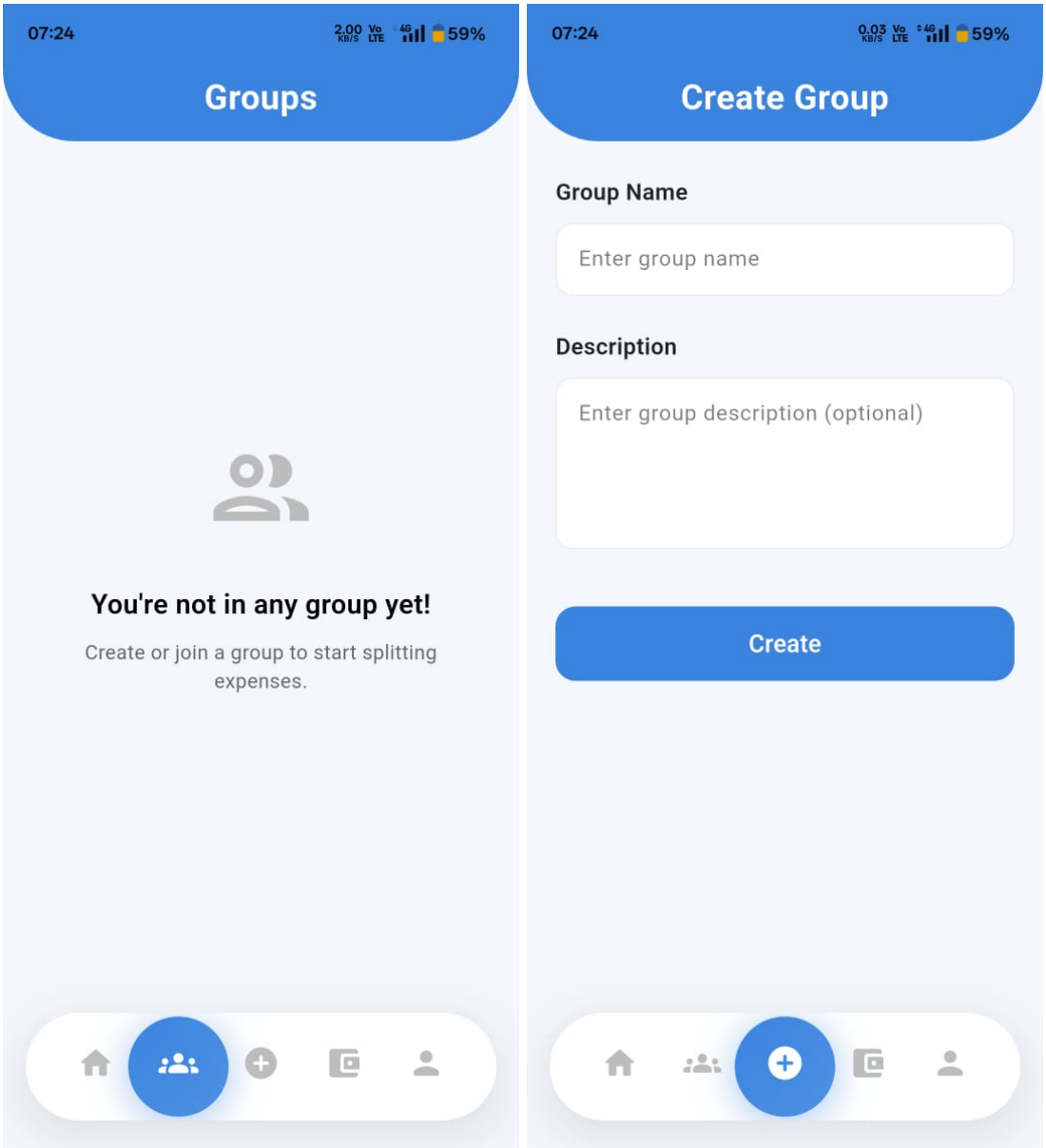

Figure 8: Empty Groups and Create group

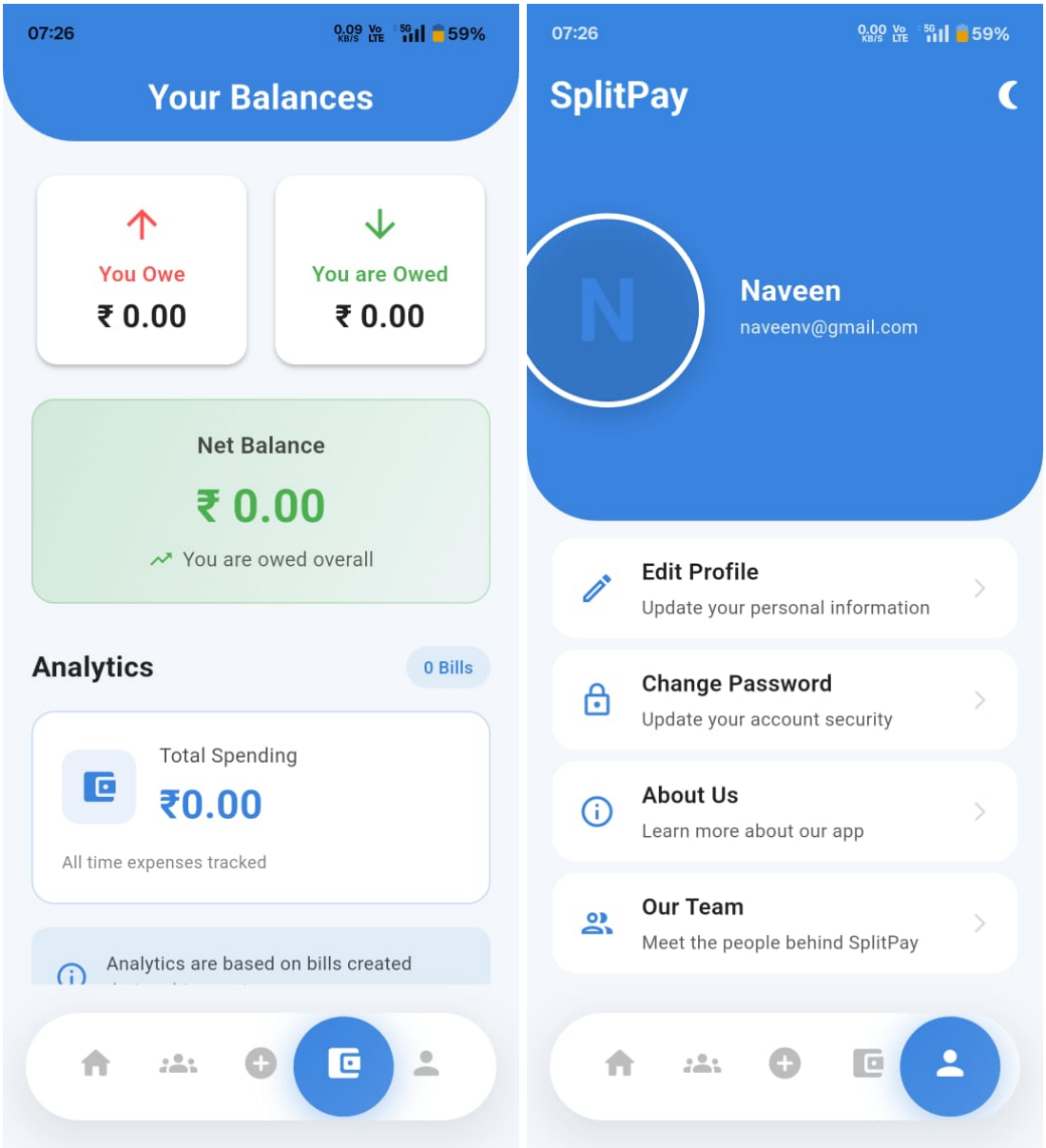

Figure 9: Balances and Profile Section

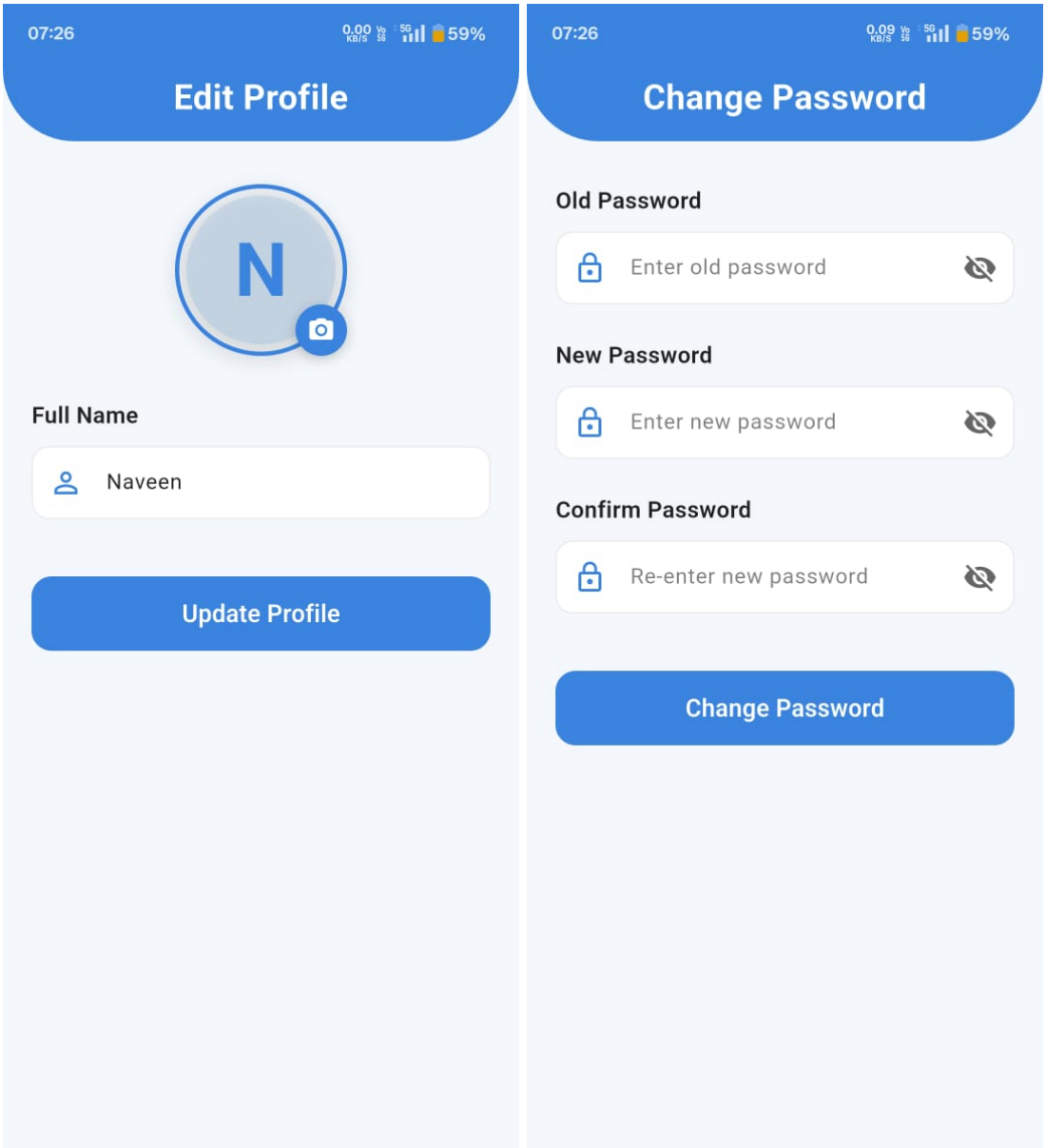

Figure 10: Edit Profile and Change Password

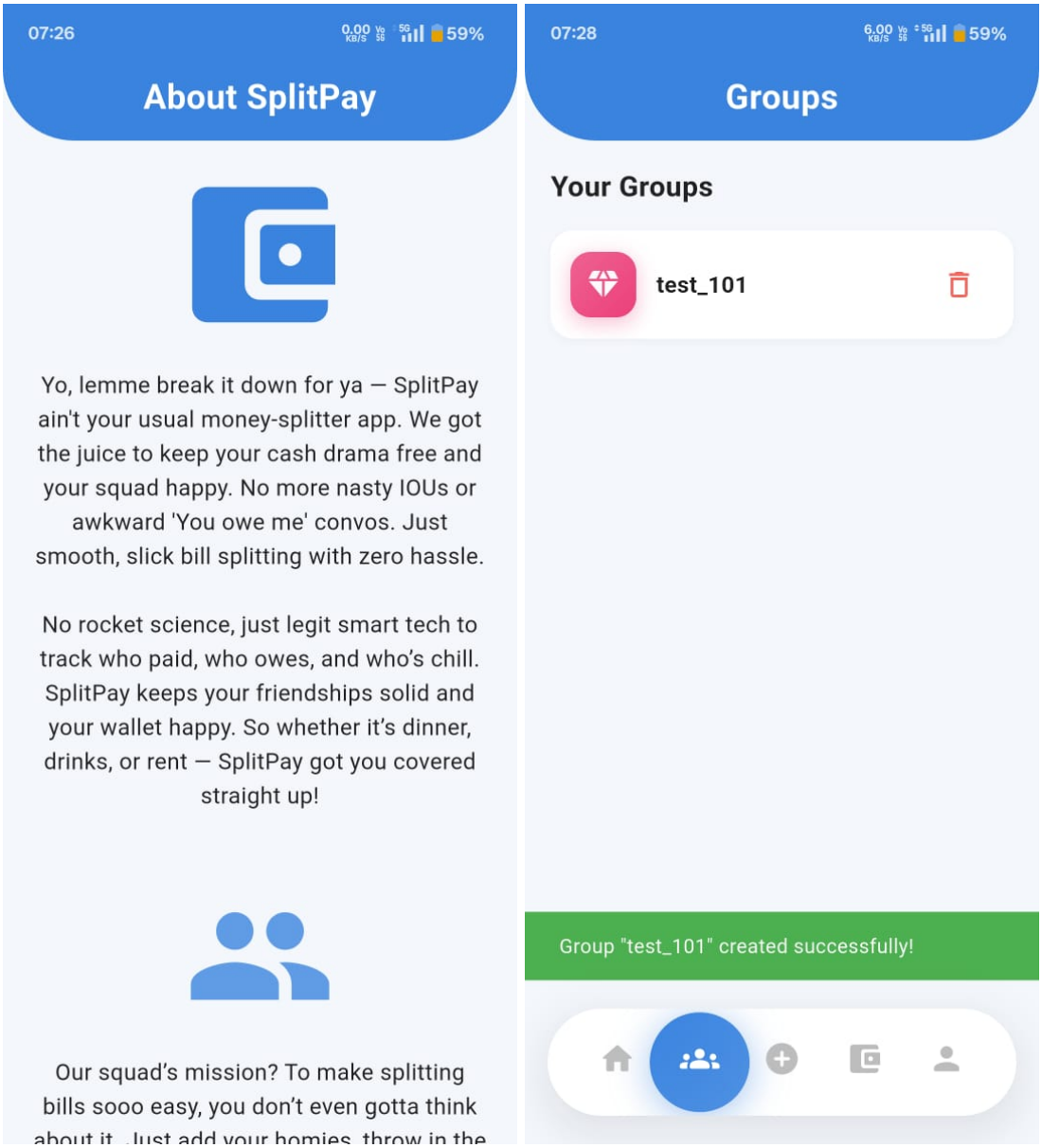

Figure 11: About us and Groups page

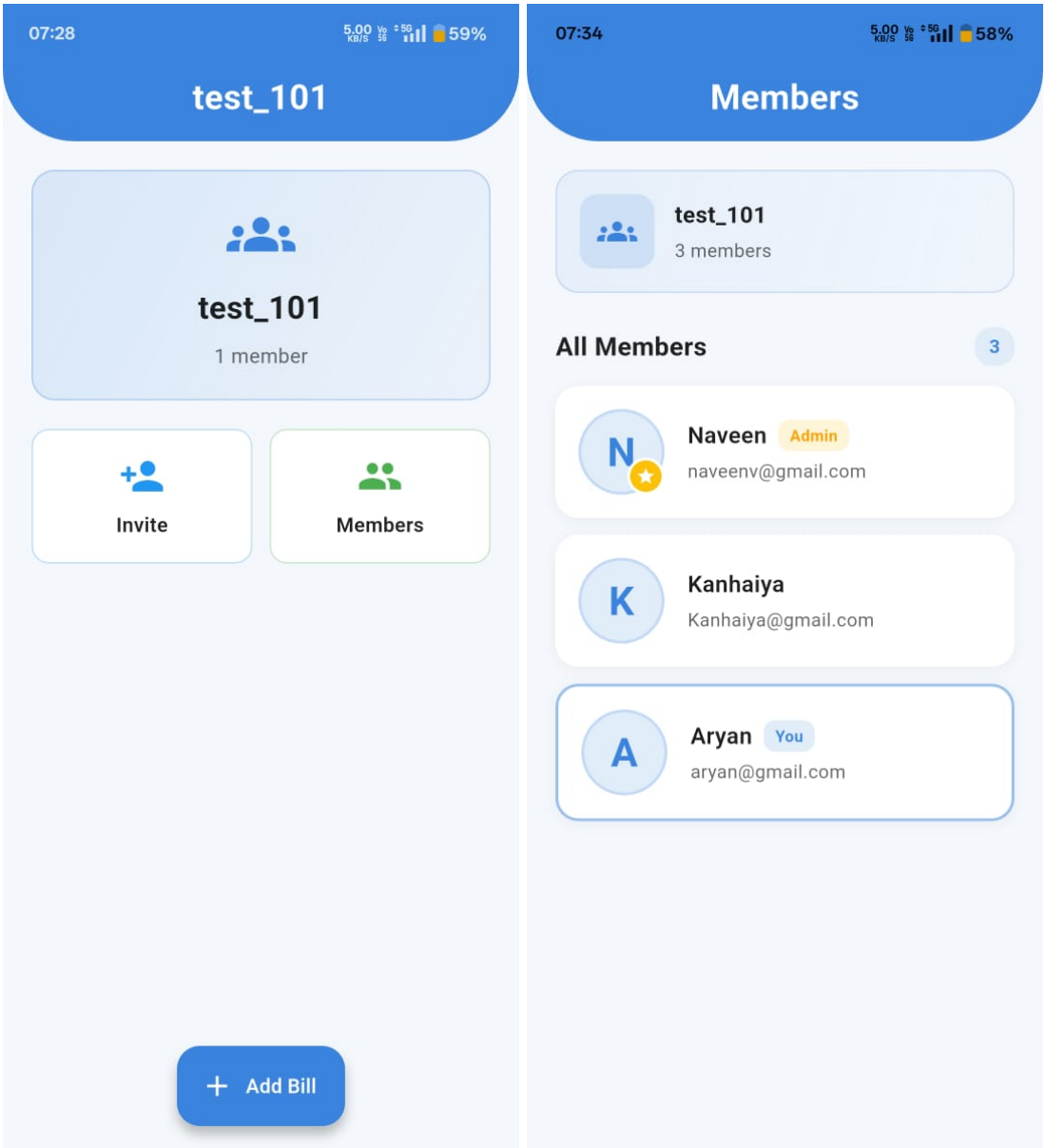

Figure 12: Selected Group and Group Members

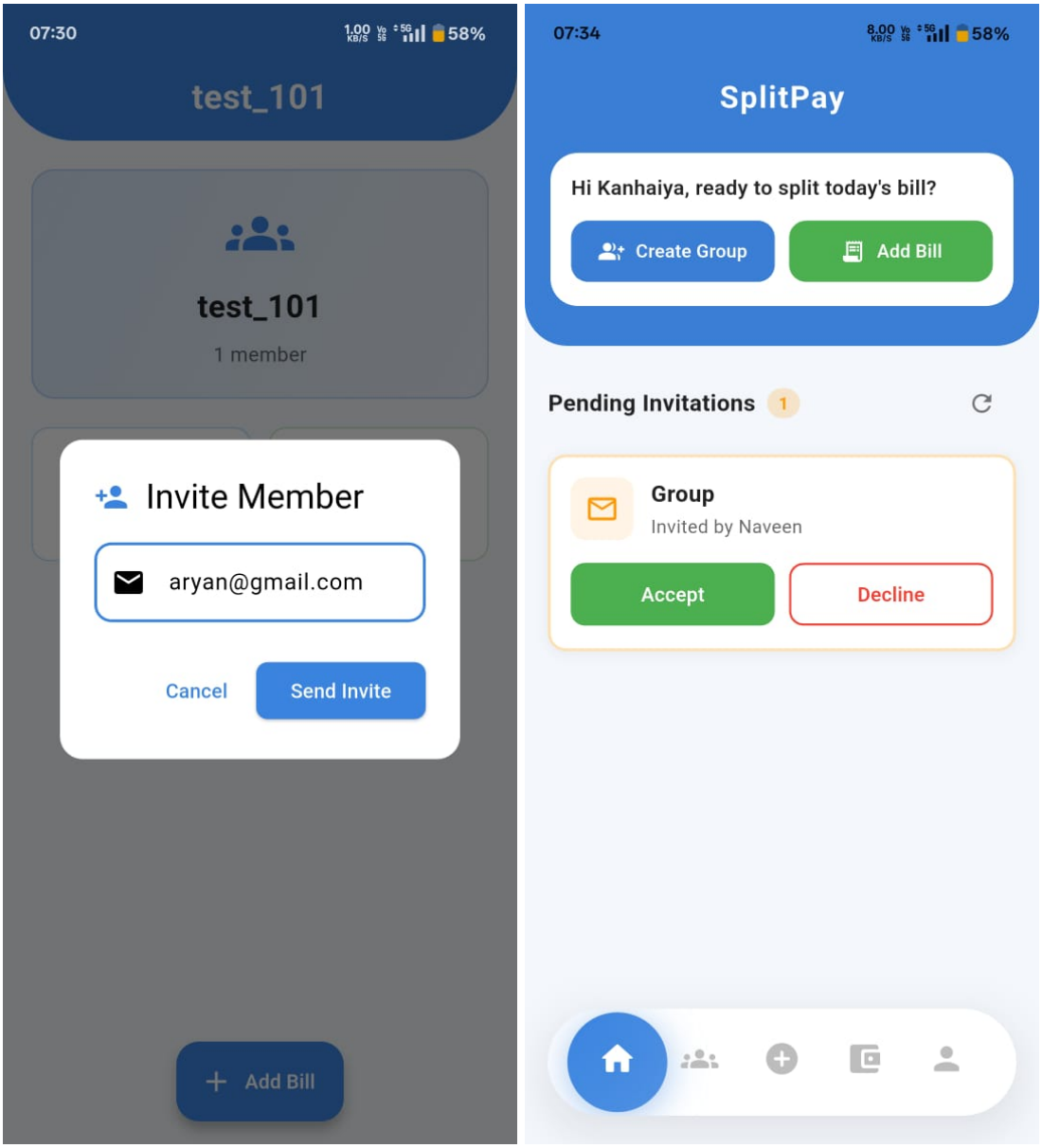

Figure 13: Group Invite and Receiving Invitations

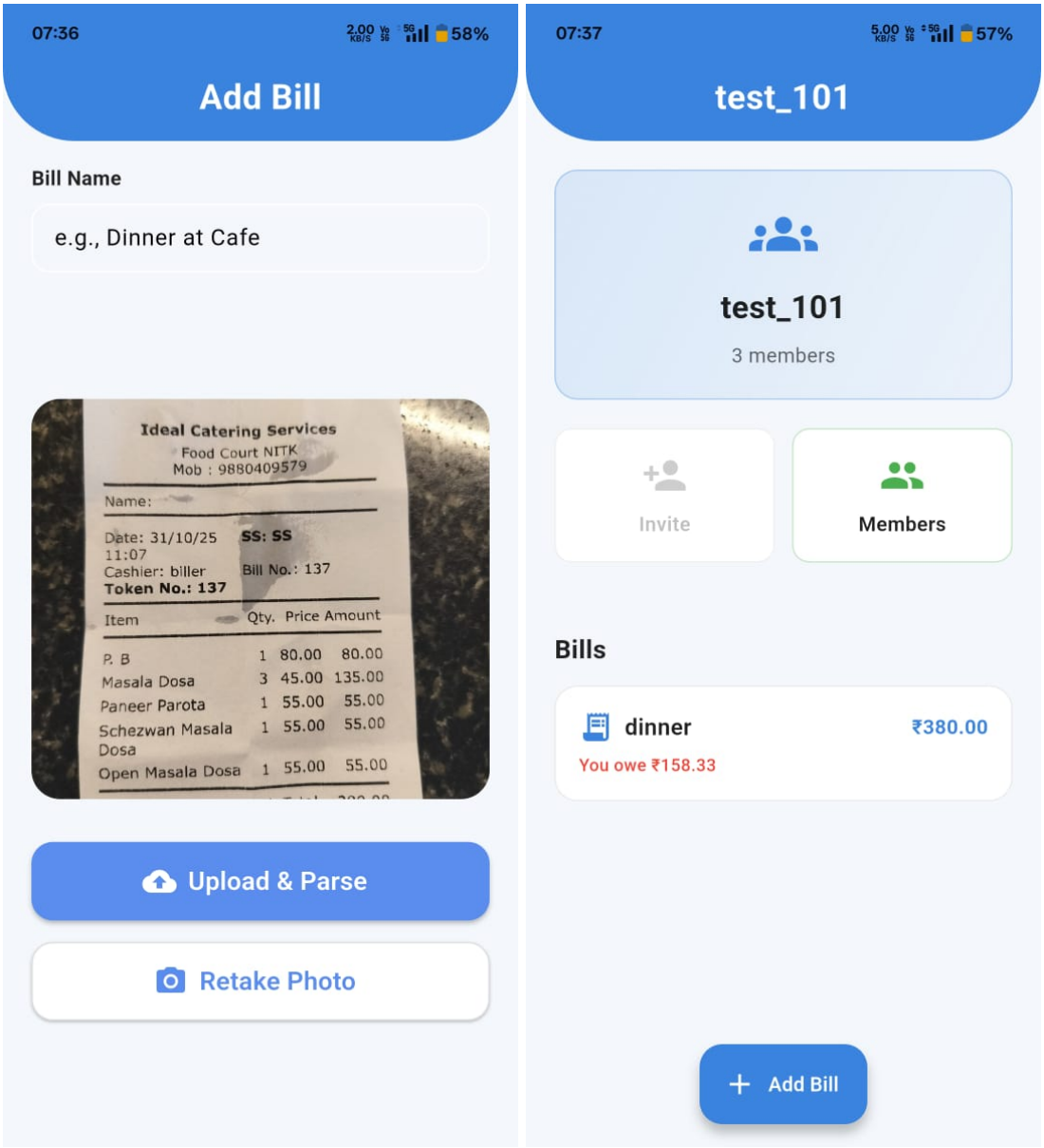

Figure 14: Add bill page and when it is added

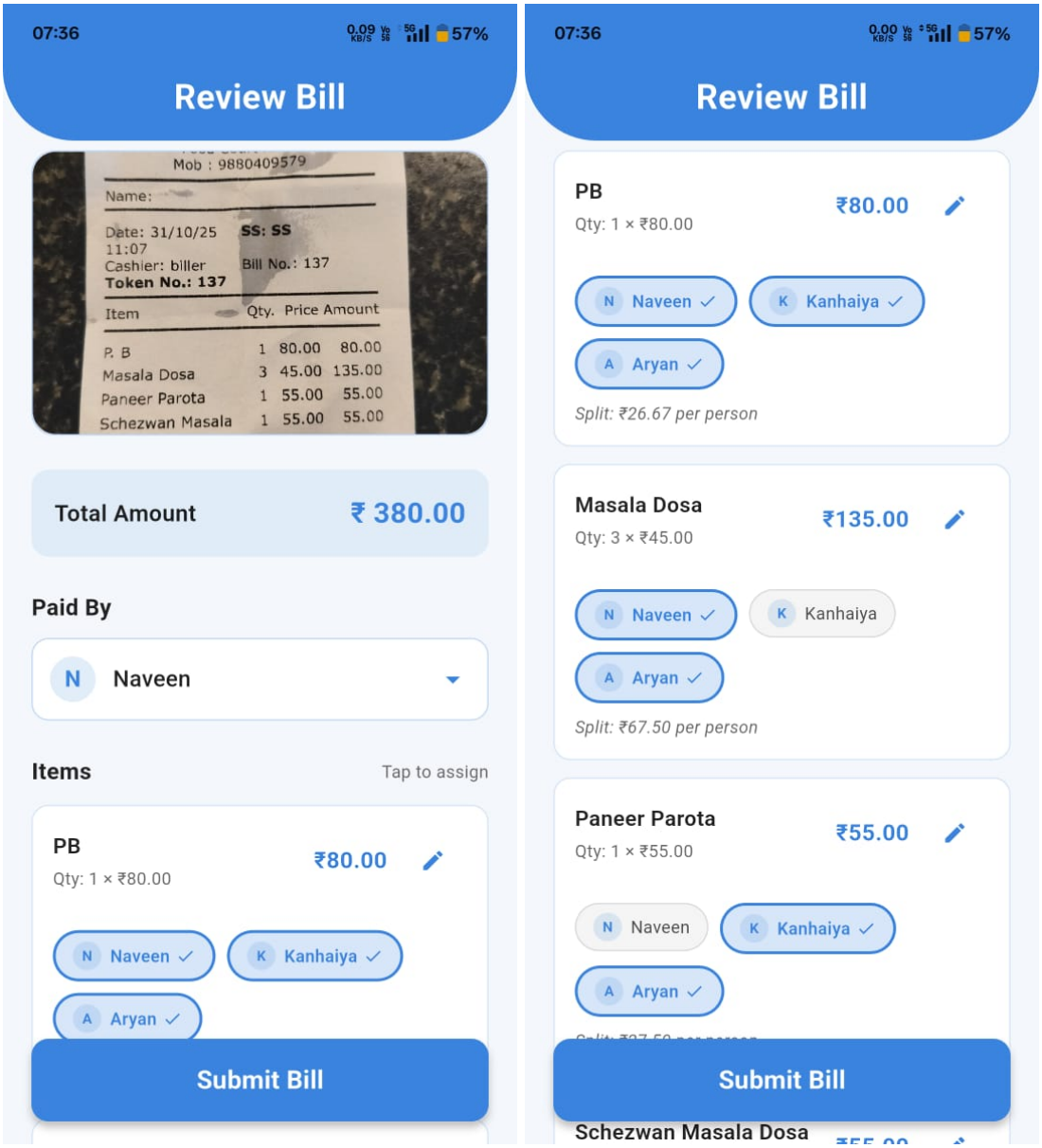

Figure 15: Reviewing Bill

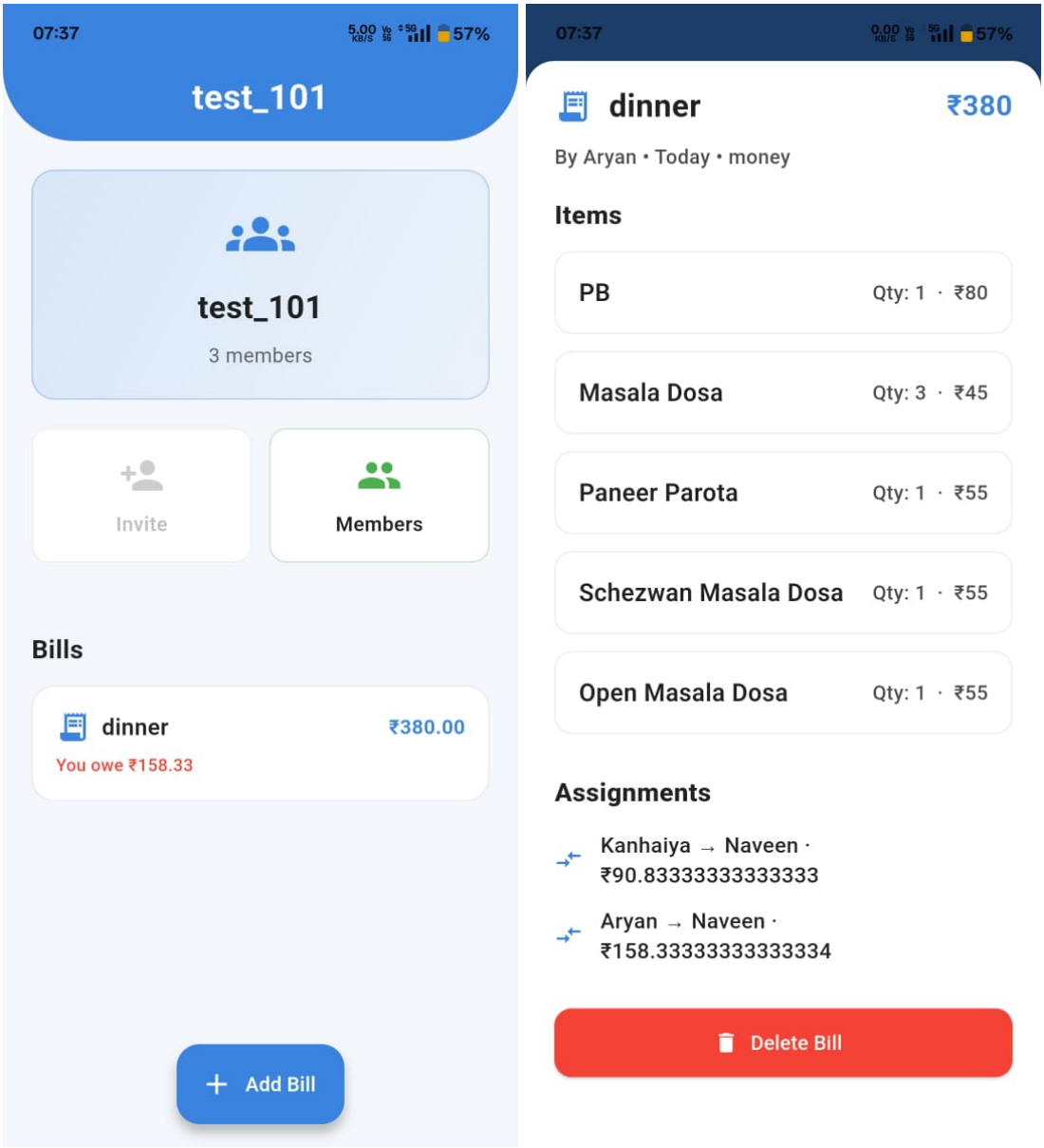

Figure 16: After bill group page

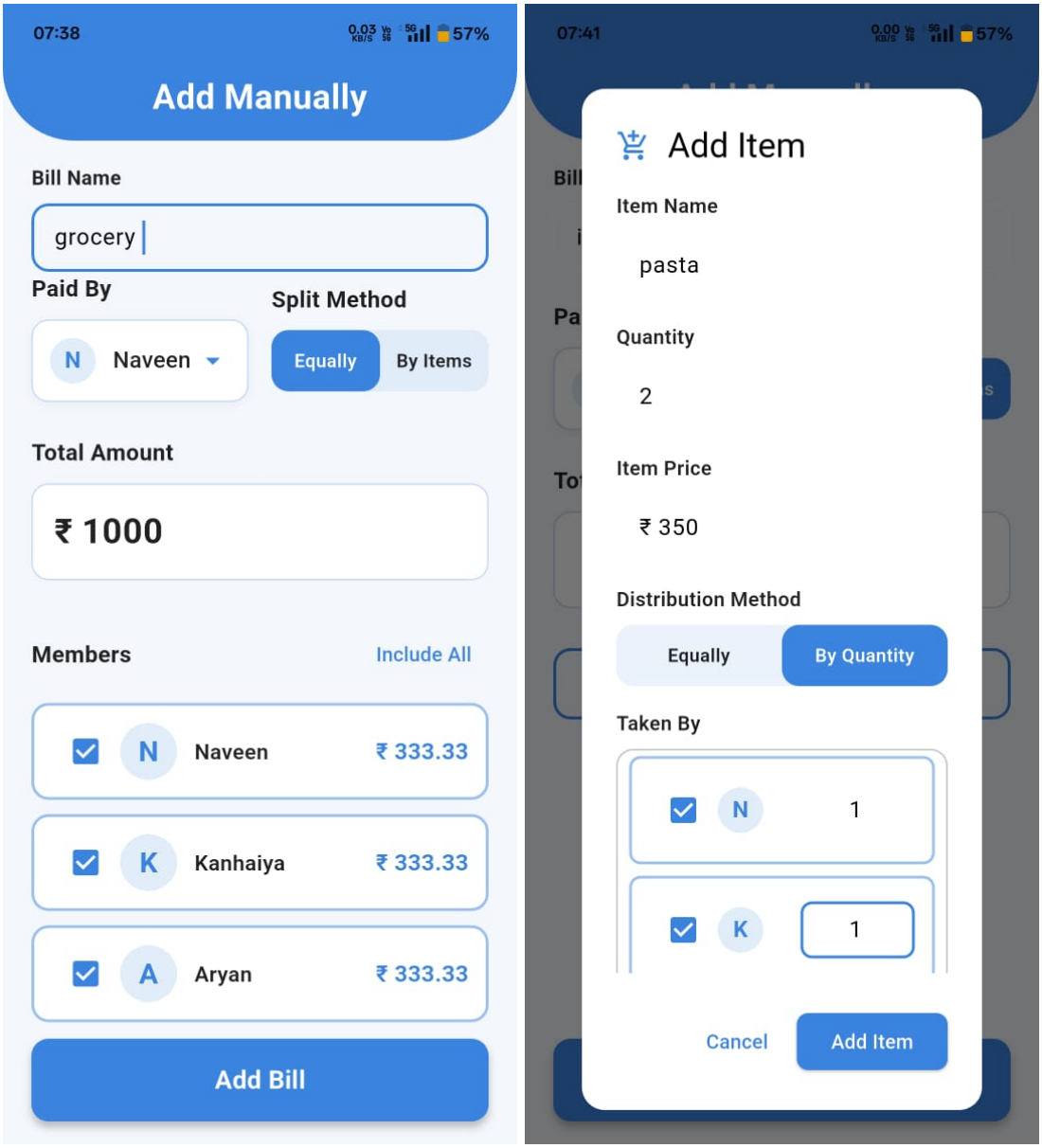

Figure 17: Add Manually

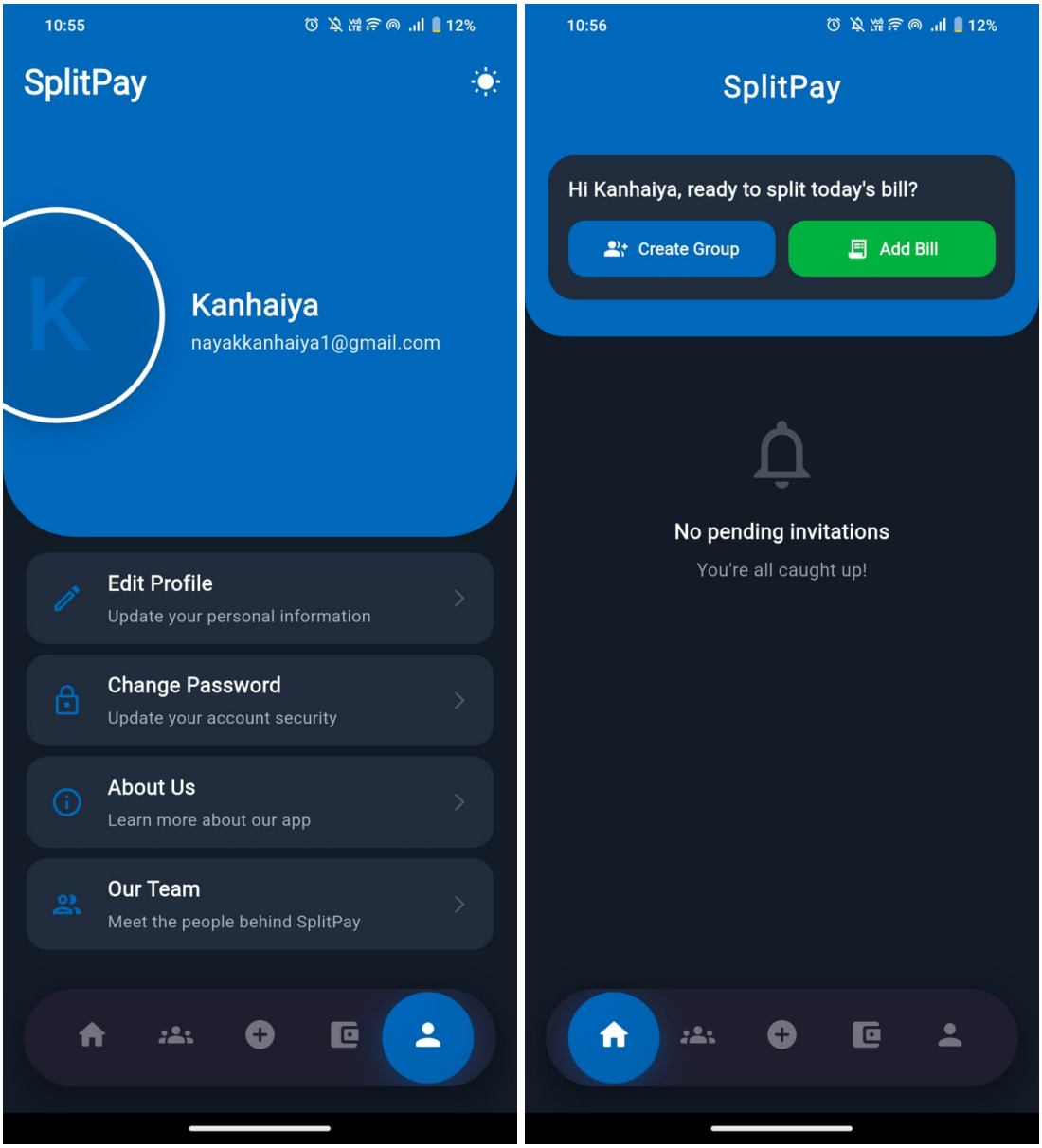

Figure 18: Dark Mode Theme

19. References

- OpenAI GPT-4o Documentation: https://platform.openai.com/docs/guides/gpt
- Ollama – Run LLMs locally: https://ollama.com/
- Tesseract.js (OCR Engine): https://github.com/naptha/tesseract.js
- Google Cloud Vision API: https://cloud.google.com/vision
- Flutter Documentation: https://docs.flutter.dev
- Node.js Documentation: https://nodejs.org/en/docs
- Express.js Guide: https://expressjs.com/en/starter/installing.html
- MongoDB Docs: https://www.mongodb.com/docs/
- Cyclic Serverless Hosting: https://docs.cyclic.sh/
- Splitwise UX Analysis: https://www.splitwise.com/
- Human-Centered Design Concepts (IDEO): https://www.ideou.com/pages/design-thinking